CERA- Architecture

Data model (MongoDB collections & schemas)

users:

```
{
  _id: ObjectId,
  name: String,
  email: String,              // unique
  phone: String,
  passwordHash: String,       // bcrypt
  role: String,               // 'resident'|'volunteer'|'coordinator'|'admin'
  skills: [String],           // e.g., ['first_aid','search_rescue']
  certified: Boolean,         // volunteer approved?
  location: {                 // last known location (GeoJSON)
    type: "Point",
    coordinates: [lng, lat]
  },
  pushTokens: [{platform: String, token: String}],
  createdAt: Date,
  updatedAt: Date
}
```

incidents:

```
{
  _id: ObjectId,
  reporterId: ObjectId,
  title: String,
  description: String,
  type: String,               // 'medical','fire','flood','infra'
  severity: String,           // 'low'|'medium'|'high'
  location: { type: "Point", coordinates: [lng, lat] },
  mediaUrls: [String],        // signed URLs
  status: String,             // 'reported'|'verified'|'assigned'|'resolved'|'closed'
  assignedTo: [ObjectId],     // volunteers assigned
  createdAt: Date,
  updatedAt: Date,
  approvedBy: ObjectId        // coordinator who verified
}
```

assignments:
```
{
  _id: ObjectId,
  incidentId: ObjectId,
  volunteerId: ObjectId,
  status: String,            // 'pending'|'accepted'|'declined'|'in_progress'|'completed'
  assignedAt: Date,
  respondedAt: Date,
  completedAt: Date
}
```

logs:
```
{
  event: String,
  payload: {},
  createdAt: Date
}
```

Indexes

- users.location 2dsphere
- incidents.location 2dsphere
- incidents.status for quick filters
- incidents.createdAt for time-based queries

API design (sample endpoints)

Auth & Users

- **POST** /api/auth/register — body: {name,email,phone,password,role,skills}

  If role = volunteer, certified=false until coordinator approves.

Returns: { token, user }

- **POST** /api/auth/login — body: {email,password} => returns JWT
- **GET** /api/users/me — header: Authorization: Bearer <token>
- **POST** /api/users/:id/approve — coordinator only {certified=true} (approve volunteer).
- **PATCH** /api/users/me/location (auth)

  Body: { lat, lng } → updates last-known location.

Incidents

- **POST** /api/incidents — report incident (form-data for photo)
  body: {title,description,type,severity,location{lat,lng},media}
- **GET** /api/incidents — list incidents (filters: type, status, bbox)
- **GET** /api/incidents/:id
- **PUT** /api/incidents/:id/verify — coordinator verifies
- **PUT** /api/incidents/:id/status — update status

Assignments / Dispatch

- **POST** /api/incidents/:id/dispatch— trigger dispatch algorithm (coordinator or system)
- **POST** /api/assignments/:id/respond — volunteer response (accept/decline)
- **GET** /api/assignments — volunteer tasks

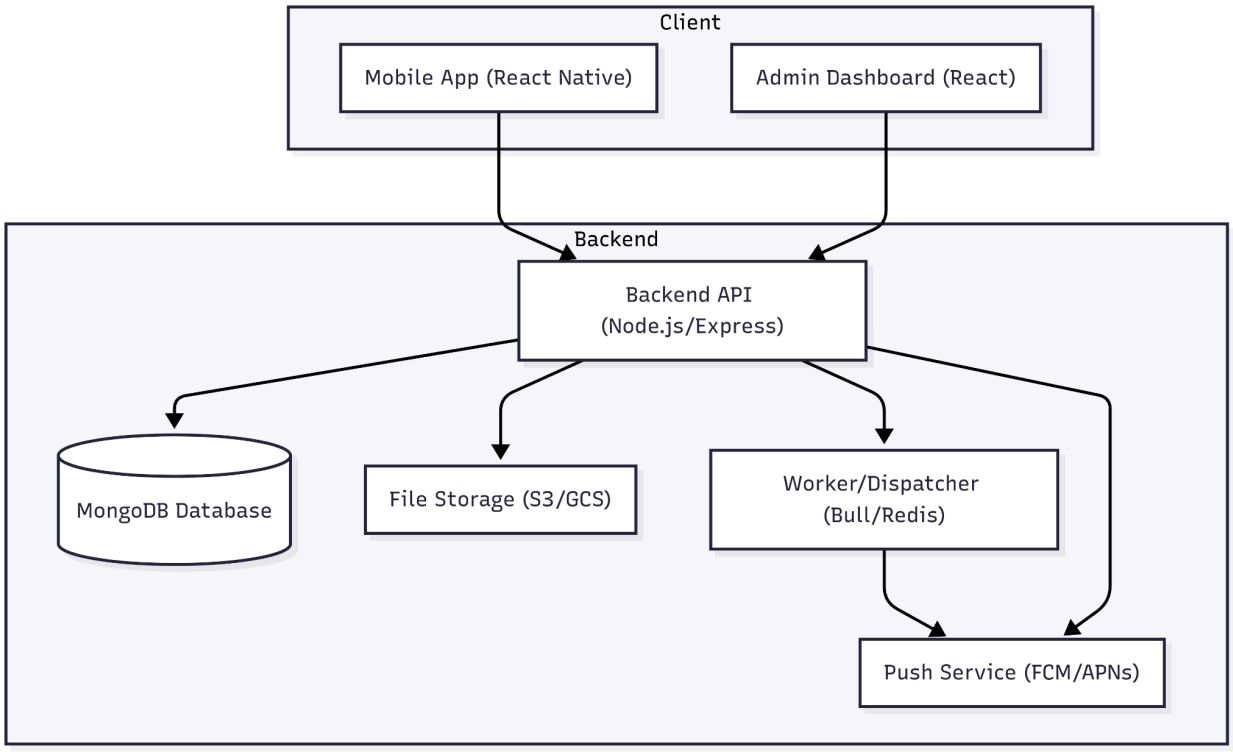
Push / Notifications

- **POST** /api/users/me/pushtoken — register device token
- **GET** /api/notifications — list notifications

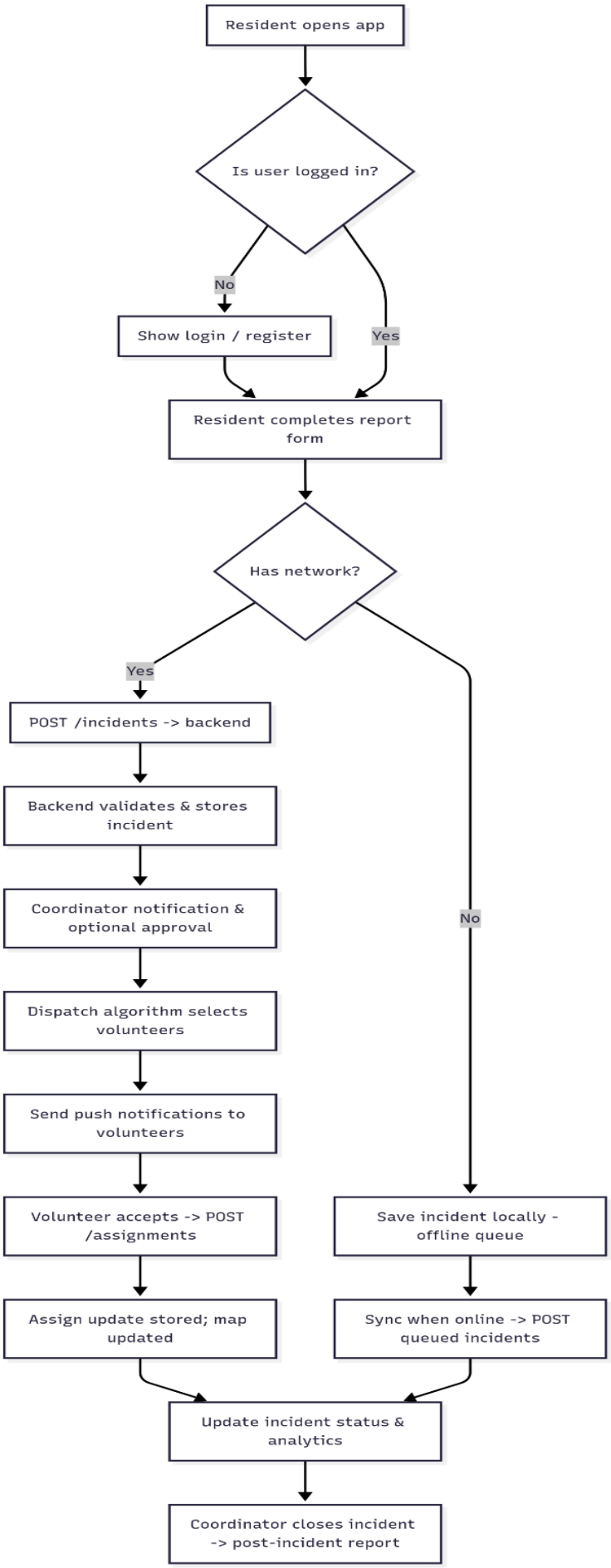
Admin / Analytics

- **GET** /api/analytics/summary — counts, response times, maps summary

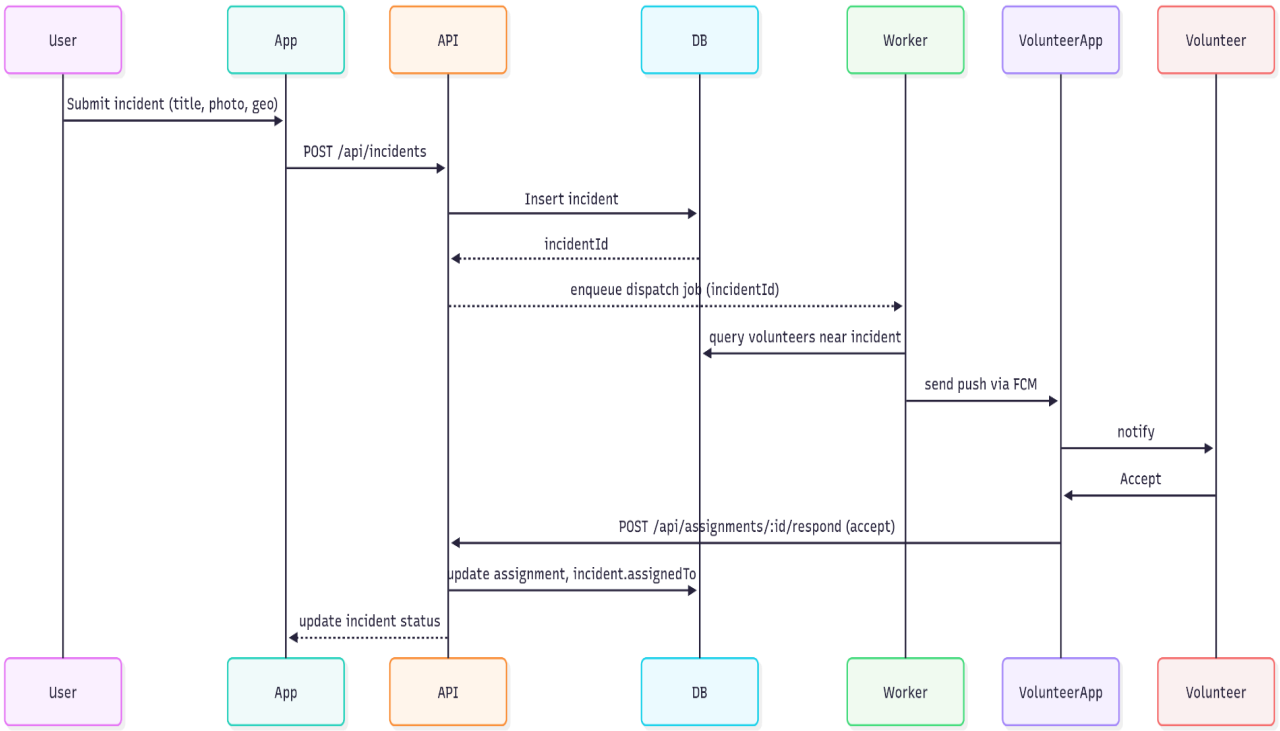
System Architecture



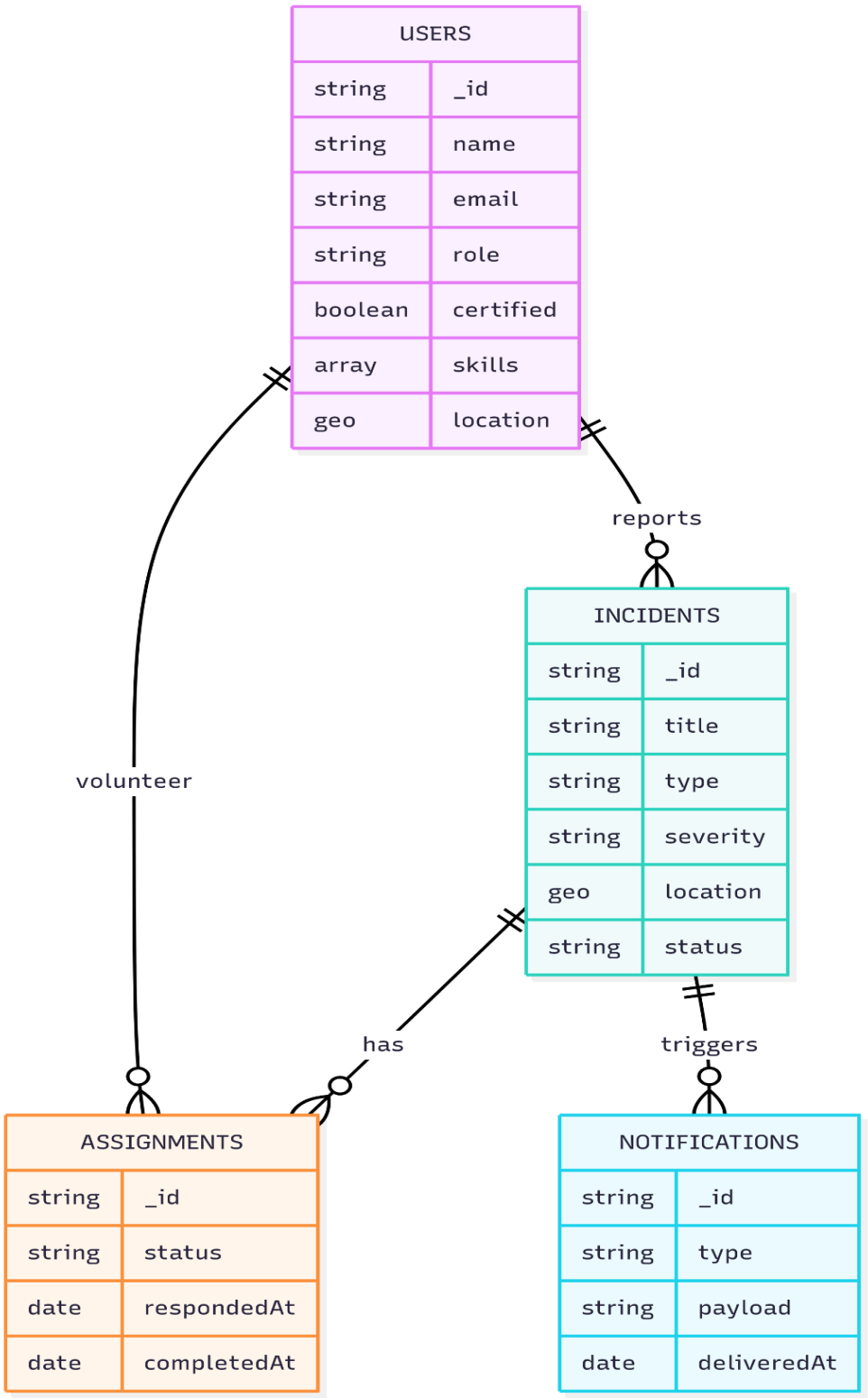
Work Flow Diagram


Resident opens app
Is user logged in?
No
Show login / register
Yes
Resident completes report form
Has network?
Yes
POST /incidents -> backend
Backend validates & stores incident
Coordinator notification & optional approval
Dispatch algorithm selects volunteers
Send push notifications to volunteers
Volunteer accepts -> POST /assignments
Assign update stored; map updated
No
Save incident locally - offline queue
Sync when online -> POST queued incidents
Update incident status & analytics
Coordinator closes incident -> post-incident report

Sequence Diagram



Data Relation Diagram



Deployment Diagram